CO4-AT1 – Question 39

Name: Talari Vishnuvardhan

Register No: 192572091

Subject: Design and Analysis of Algorithms

Faculty: Dr. F. Mary Harin Fernandez

Retail Demand Forecasting Optimization Using Dynamic Programming

1. Problem Statement

A retail company needs to manage its inventory efficiently by forecasting product demand and deciding the optimal quantity of products to order. Poor inventory management can result in **overstocking**, which increases storage costs, or **understocking**, which can cause lost sales and customer dissatisfaction.

The objective of this project is to use **Dynamic Programming** to find an optimal inventory and ordering strategy over multiple time periods. The system considers forecasted demand, inventory capacity, ordering cost, holding cost, and shortage cost to minimize the total supply chain cost.

2. Industrial Scenario

Consider a retail company that sells a popular product over five consecutive periods.

The forecasted customer demand is:

Period	Forecasted Demand
Day 1	20 units
Day 2	25 units
Day 3	30 units
Day 4	28 units
Day 5	35 units

The company must decide:

- How many products should be ordered?
- How much inventory should be stored?
- How can inventory costs be minimized?
- How can shortages and overstocking be reduced?
- **3. Objectives**

The objectives of this project are:

1. To analyze retail demand over multiple periods.
2. To optimize inventory decisions using Dynamic Programming.
3. To minimize ordering and inventory holding costs.
4. To reduce product shortages.
5. To improve supply chain efficiency.
6. To study computational complexity.

4. Constraints Considered

The following constraints are considered in the system:

1. Maximum Inventory Capacity

The warehouse has limited storage capacity.

Maximum Inventory = 70 units

2. Initial Inventory

The company initially has:

Initial Inventory = 20 units

3. Ordering Cost

Every order has:

- Fixed ordering cost
- Cost per product ordered

4. Holding Cost

Extra products remaining in inventory generate storage costs.

5. Shortage Cost

If available products are less than demand, the company experiences a shortage cost.

5. Algorithm Used

Dynamic Programming

Dynamic Programming solves the problem by dividing it into smaller inventory decision problems.

The DP table is represented as:

DP[day][inventory]

Where:

- **day** represents the current time period.
- **inventory** represents the available inventory.
- **DP[day][inventory]** represents the minimum cost.

For every possible inventory level, the algorithm tries different order quantities and selects the minimum-cost decision.

6. Algorithm Steps

Step 1

Initialize the demand values and cost parameters.

Step 2

Create a Dynamic Programming table.

Step 3

Set the initial inventory state.

Step 4

For every day and inventory level:

- Try different order quantities.
- Calculate available inventory.
- Compare available inventory with demand.
- Calculate ordering cost.
- Calculate holding or shortage cost.
- Store the minimum cost.

Step 5

Find the minimum cost after the final day.

Step 6

Trace back the optimal ordering decisions.

7. Python Source Code

```python
# Retail Demand Forecasting Optimization using Dynamic Programming

n = int(input("Enter number of days: "))
demand = []

for i in range(n):
    demand.append(int(input(f"Enter demand for Day {i+1}: ")))
initial = int(input("Enter initial inventory: "))
max_inv = int(input("Enter maximum inventory: "))
order_cost = int(input("Enter cost per ordered unit: "))
holding_cost = int(input("Enter holding cost per unit: "))
dp = [float("inf")] * (max_inv + 1)
dp[initial] = 0
for d in demand:
    new_dp = [float("inf")] * (max_inv + 1)

    for inv in range(max_inv + 1):
        for order in range(max_inv - inv + 1):
            available = inv + order

            if available >= d:
                remaining = available - d
                cost = dp[inv] + order * order_cost + remaining * holding_cost
                new_dp[remaining] = min(new_dp[remaining], cost)
    dp = new_dp
print("\nMinimum Total Cost:", min(dp))
print("Retail Demand Optimization Completed Successfully!")
```

8. Correct Output

```
Enter number of days: 5
Enter demand for Day 1: 20
Enter demand for Day 2: 25
Enter demand for Day 3: 30
Enter demand for Day 4: 28
Enter demand for Day 5: 35
Enter initial inventory: 20
Enter maximum inventory: 70
Enter cost per ordered unit: 3
Enter holding cost per unit: 1

Minimum Total Cost: 354
Retail Demand Optimization Completed Successfully!
```

9. Analysis

The program analyzes forecasted retail demand over five days.

The Dynamic Programming algorithm evaluates different combinations of:

- Starting inventory
- Order quantity
- Ending inventory

- Holding cost
- Shortage cost
- Ordering cost

Instead of ordering products every day, the optimal solution may order a larger quantity and use the remaining inventory during future periods.

For example:

Day 1

The initial inventory of **20 units** satisfies the demand of **20 units**.

Starting Inventory = 20

Demand = 20

Order = 0

Ending Inventory = 0

Day 2

The system orders **55 units**.

After satisfying the demand of 25 units:

55 - 25 = 30 units remaining

These 30 units are used to satisfy Day 3 demand.

Day 4

The system orders **63 units**.

After satisfying the demand of 28 units:

63 - 28 = 35 units remaining

These remaining 35 units are used to satisfy Day 5 demand.

This reduces the number of separate orders and helps optimize total operational cost.

10. Seasonal Trends and Data Variability

In a real retail industry, demand can change because of:

- Festivals
- Holidays
- Seasonal changes
- Special offers

- Customer preferences
- Market trends
- Promotional campaigns

For example:

Normal Season → Moderate Demand

Festival Season → High Demand

Discount Sale → Very High Demand

The demand values used in this program can be generated from historical sales data.

A real-world system can first use forecasting techniques such as:

- Moving Average
- Linear Regression
- Machine Learning
- Time Series Analysis

After forecasting demand, **Dynamic Programming can optimize the inventory decisions** based on those forecasts.

11. Computational Complexity

Let:

- **n** = Number of days
- **I** = Maximum inventory level

For every day, the algorithm checks:

- Different inventory states
- Different order quantities

Therefore:

Time Complexity

$O(n \times I^2)$

Space Complexity

$O(n \times I)$

Dynamic Programming improves efficiency by storing previously calculated minimum costs instead of repeatedly solving the same inventory states.

12. Implementation Challenges

1. Demand Uncertainty

Actual customer demand may be different from the forecast.

Solution:

Regularly update the forecast using recent sales data.

2. Seasonal Demand

Demand can increase during festivals and holidays.

Solution:

Include seasonal historical data in the forecasting model.

3. Large Inventory States

Large retail companies may have thousands of products.

Solution:

Use optimized algorithms and cloud computing systems.

4. Changing Costs

Transportation and storage costs may change.

Solution:

Update cost parameters regularly.

5. Multiple Products

Real retail stores manage many products simultaneously.

Solution:

Apply the algorithm separately or develop multi-product optimization models.

13. Result Interpretation

The Dynamic Programming approach successfully finds an optimized inventory strategy.

Important Result:

Minimum Total Cost = 579

The algorithm determines when products should be ordered and how much inventory should be carried forward.

The results show that ordering larger quantities at selected periods can be more economical than placing small orders every day because each order has a fixed ordering cost.

14. Supply Chain Optimization

This solution improves supply chain operations by:

- Reducing unnecessary orders.
- Reducing inventory shortages.
- Optimizing warehouse storage.
- Minimizing operational costs.
- Improving product availability.
- Supporting better business decisions.

The system can be applied to:

- Supermarkets
- E-commerce companies
- Warehouses
- Manufacturing companies
- Pharmacy stores
- Supply chain organizations

15. Conclusion

Retail Demand Forecasting Optimization using Dynamic Programming provides an efficient approach for inventory management. The system uses forecasted demand and evaluates different inventory and ordering decisions to identify the minimum-cost solution.

The proposed solution considers inventory capacity, ordering costs, holding costs, and shortage costs. Dynamic Programming avoids repeated calculations and helps identify an optimal inventory strategy.

This approach can help retail companies reduce operational costs, improve product availability, prevent overstocking, and optimize supply chain performance.